

GRIDARENA LLM System: A Natural Language Interface Framework for Domain-Specific APIs

Sebastian Sjøen-Tollaksvik
INESCTEC

Faculdade de Ciências do Universidade porto (FCUP)
Porto, Portugal
up202503018@up.pt

Abstract—

This paper presents a framework for natural language interfaces for domain-specific APIs, built and validated on the gridarena low-voltage grid analysis platform. The system allows engineers to interact with complex REST APIs through plain English, without requiring software development knowledge. A five-layer architecture combines LLM tool calling, a per-agent retrieval-augmented generation pipeline (RAG), and a two-stage response format that separates tool execution from domain-aware interpretation. The design is domain-independent: routing, retrieval, conversation history, and mock fallback infrastructure are reusable across any target API, with domain adaptation requiring only documentation authorship and agent wrappers.

*Index Terms—*large language models, retrieval-augmented generation, API integration, natural language interface, tool calling

I. INTRODUCTION

The gridarena platform gives engineers access to specialised computational tools for low-voltage grid analysis through a REST API. Using it requires knowing which endpoints to call, in what order, with what parameters, and how to interpret the responses, granting electrical engineers software development knowledge they reasonably lack.

This project builds a natural language layer on top of that API. Users only need to type plain English. The system determines whether a tool call is required, executes it if so, and returns both the raw result and a domain-aware interpretation of what it means.

The architecture is a general-purpose framework for wrapping any domain-specific API with a conversational interface. The five-layer design, uniform agent pattern, per-collection RAG pipeline, and structured mock fallback are all domain-agnostic. Gridarena is where the framework was built and validated. It is not where it ends. A demonstration of the system in use is available here.

II. STATE OF THE ART

A. LLM Tool Use and Function Calling

In June 2023, OpenAI introduced function calling for GPT-4 and GPT-3.5-Turbo, allowing developers to describe functions to the model and have it intelligently output a JSON object containing arguments to call those functions [4]. The capability spread rapidly across providers, and the Model Context

Protocol, introduced by Anthropic in November 2024 and subsequently adopted by both OpenAI and Google, established a cross-provider standard for connecting AI models to external tools and data sources [2]. The mechanism works by providing the model with function definitions in JSON Schema format, where the model calls predefined functions with structured parameters, enabling interaction with external tools and APIs [1]. These APIs prescribe the tool-calling mechanism itself but leave architectural decisions: how to route calls, how to inject domain knowledge, how to interpret results entirely to the developer.

B. Retrieval-Augmented Generation

Large language models store knowledge in their parameters at training time, which creates a fundamental limitation: they cannot access domain-specific, proprietary, or updated information at inference time. RAG addresses this by combining pre-trained parametric memory with non-parametric memory, using a dense vector index to inject relevant documents at query time [3]. This concept was first introduced in 2020 and demonstrated that RAG models generate more specific, diverse, and factual language than parametric-only baselines [3]. In practice, RAG pipelines embed documentation into a vector store at startup and retrieve the most semantically similar chunks at query time, injecting them into the model's context window. This allows a general-purpose model to answer questions grounded in documentation it was never trained on, making it applicable to any specialised domain [3].

C. LLM-Based API Agents

Several systems have explored connecting LLMs directly to real-world APIs through natural language. Gorilla [5] demonstrated that integrating a retrieval system with an LLM substantially mitigates hallucination and enables the model to keep up with frequently updated API documentation. However, Gorilla's approach requires fine-tuning a dedicated model on API corpora, making it expensive to adapt to a new domain. RestGPT [6] takes a different approach: it connects LLMs with RESTful APIs and tackles the practical challenges of planning, API calling, and response parsing through a coarse-to-fine online planning framework. RestGPT consists of a Planner,

an API Selector, and an Executor, each driven by prompting an LLM [6].

Both systems are built and evaluated for specific API domains and do not provide a transferable framework for deploying the same architecture against a new API without significant re-engineering. The system described in this report addresses that gap directly: it is designed as a domain-independent framework in which adding a new API requires only writing domain documentation and agent wrappers, with routing, retrieval, and fallback infrastructure already in place.

III. SYSTEM ARCHITECTURE

The system is built around two guiding principles: each component has one responsibility and communicates with the rest through narrow, well-defined interfaces, and the model is always given one focused task at a time rather than multiple competing objectives. The first makes the architecture extensible by design. The second reduces the surface area for errors and hallucination. The following subsections describe each layer in detail.

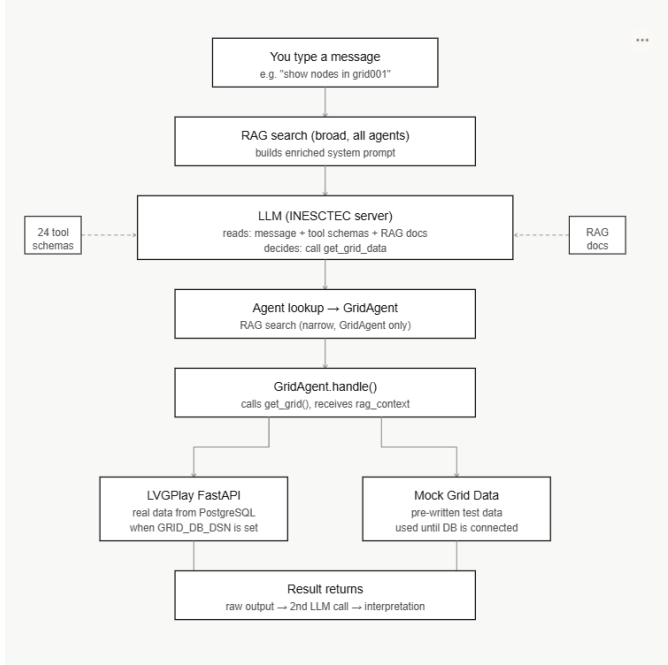


Fig. 1. Example of workflow for the query “show nodes in grid001”.

A. Layered Design

The system is divided into five layers. Each has one job, and none of them knows how the others work internally.

The flow is the same for every request. The engineer’s message comes in. The dispatcher retrieves relevant documentation from the vector database. The model sees the message, the tool definitions, and that documentation together, and responds with a structured tool selection. The dispatcher routes that selection to the right agent. The agent runs the platform function and returns a result. That raw result is displayed

TABLE I
SYSTEM LAYERS AND RESPONSIBILITIES

Layer	Responsibility
Language model	Receives messages, selects tools, produces interpretations
Tool schemas	Defines available functions as structured JSON for the model
Dispatcher	Routes tool calls to agents and orchestrates retrieval
Agent files	Wrap API calls with state tracking and mock fallback
RAG pipeline	Retrieves domain documentation and injects it into model context

immediately. A second model call then produces a plain English interpretation.

Because the layers only communicate through narrow, well-defined interfaces, any of them can change without forcing changes elsewhere. The model can be swapped out. New agents can be added. Documentation can be improved. None of that requires touching the rest of the system.

B. Agent Pattern

Gridarena is a digital environment developed at INESC TEC for interacting with low-voltage grid data, supporting grid management, historical data handling, power flow simulation, and benchmark-based validation of algorithms for phase identification, topology discovery, state estimation, and voltage control. Participants interact with the platform through a REST API to manage grid definitions, upload measurement data, run simulations, submit algorithm outputs, and retrieve benchmark scores.

TABLE II
AGENTS AND RESPONSIBILITIES

Agent	Responsibility
Grid agent	Grid registration, deletion, node and connection queries
Historical agent	Historical measurement upload, deletion, and retrieval
Phase agent	Phase identification training, testing, submission, and scoring
Topology agent	Topology discovery training, testing, submission, and scoring
State agent	State estimation training data, tasks, and scoring
Voltage agent	Voltage control scenarios, testing, submission, and scoring
Powerflow agent	Power flow execution and result retrieval

Seven agents cover the full scope of the gridarena platform.

Every agent tracks two things: the last grid ID it queried, and a call history for the session. This is what makes mid-conversation shorthand work. When an engineer says “run that again on grid002,” the agent already knows which function ran last. Method calls go straight to the underlying platform function. Nothing in between. Each method also wraps its call in an exception handler. If no database is configured, it returns hardcoded data that matches the shape of a real API response.

The rest of the system keeps working. To switch to live data, you set an environment variable without any code changes.

Every agent follows this same pattern. Adding a new one means creating one file that follows the existing structure. The rest of the system picks it up automatically. This uniformity is what makes the framework transferable: the same pattern applies to any new API domain, not just gridarena.

C. Tool Schemas

The model communicates through a standard tool-calling protocol: it receives function definitions in JSON Schema format and responds with structured function call requests. This produces observable, debuggable intermediate steps and handles structured argument extraction without any model training.

Schemas are organised into one file per agent and merged into a single master list at startup. Each definition has a name, parameter specifications, and a description field. That description field is what determines routing reliability in practice. A description written as what an engineer wants to accomplish. The schemas also serve as the readable documentation of what the system can do, independent of any agent implementation.

D. Dispatcher

The dispatcher is the routing layer. It holds three data structures: a mapping from every tool name to the agent instance responsible for handling it, a mapping from natural language keywords to agent names for retrieval scoping, and a mapping from agent names to their documentation file paths for the embedding process.

When a tool selection arrives from the model, the dispatcher looks up the agent, constructs a natural language query from the tool name and arguments, retrieves the most relevant documentation chunks from that agent's vector database collection, and passes everything through to the agent.

Adding a new agent requires changes in exactly five places: one import, one documentation path entry, keyword entries for retrieval scoping, one agent instantiation, and tool name mappings. No conditional logic elsewhere needs updating. This is intentional. The dispatcher is designed so that extension does not require understanding the full system.

The keyword scoping keeps retrieval focused. A query about power flow results searches only power flow documentation, not phase identification or topology discovery. This reduces noise and avoids burning the context window on irrelevant material. If no keywords match, the dispatcher falls back to a default set of agents.

E. Retrieval-Augmented Generation Pipeline

The model has no built-in knowledge of the gridarena platform, and would have no built-in knowledge of any target domain. It does not know what a phase identification score of 0.33 implies, or what voltage range counts as a violation, or which operations must happen before a power flow can run. The RAG pipeline is what fills that in, and this mechanism generalises directly to any new domain.

It runs in two stages. At startup, a dedicated embedding script processes all documentation files: each is split into overlapping chunks, embedded using a sentence-transformer model, and stored in a ChromaDB collection named after the corresponding agent [7]. ChromaDB was selected because it is open source and runs locally with no external dependencies [7]. Development was carried out on a single machine with no budget for hosted services, so a self-contained vector store was a practical requirement, not just a preference. At query time, the dispatcher retrieves the most semantically similar chunks from the relevant collection and passes them to the agent alongside the tool arguments.

One collection per agent, rather than one shared collection, is intentional. A single shared collection was considered but rejected because it places too much reliance on the vector database to distinguish between structurally similar content from different domains, rather than encoding that separation cleanly in the architecture itself. A shared collection risks surfacing structurally similar but domain-irrelevant content, a state estimation chunk appearing during a topology discovery query, for instance. The per-agent design prevents that.

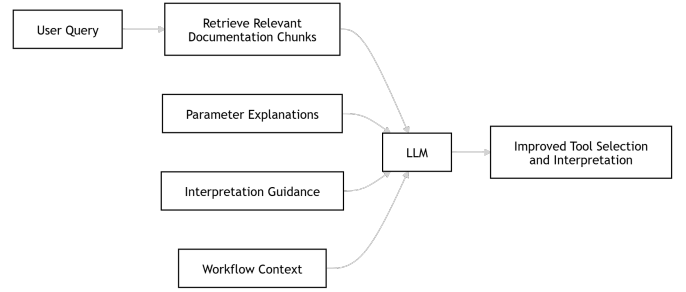


Fig. 2. Visualisation of context injection for the language model before tool selection and interpretation. Retrieved documentation chunks from the per-agent ChromaDB collection are injected alongside the tool schemas and message history

The documentation files themselves are written specifically for the model to consume. Each function section describes what the engineer wants to accomplish, explains parameters in domain terms, and includes interpretation guidance: what voltage thresholds are in physical units, what a perfect score looks like versus a random baseline, how anonymized identifiers work, and what workflow steps must precede a given operation. That last point matters most as it is what gives the model enough context to flag ordering issues without any hard-coded logic. For any new domain, writing documentation in this style is the single highest-leverage step in deploying the framework.

F. Conversation Loop

The full message history is kept across every turn in a session. Every new message appends to that history, which is sent to the model on each call. This allows the model to resolve references that span multiple turns, combined with the per-agent state tracking described above.

Every tool execution involves two separate model calls. The first is for tool selection: the model receives the full message history, all tool schemas, and the retrieved documentation context, and responds with a tool selection. The second is for interpretation: once the tool result is appended to history, the model produces a plain English explanation. Tools are not available during this second call, which prevents nested tool calls from occurring during interpretation.

This separation was a deliberate architectural decision rooted in how LLMs perform best: given one focused task at a time rather than multiple competing objectives in a single call. Combining tool selection and interpretation into one call increases the surface area for errors and hallucination. Keeping them separate gives the model a simpler, more constrained task at each step, which is the same philosophy that shapes the layered design as a whole.

Raw output is always displayed before the interpretation. Engineers can check the actual data, compare raw values against what the model says, and catch any discrepancy. The interpretation prompt explicitly instructs the model not to reproduce or reformat the raw output, it should add understanding, not restate what is already visible.

The two-stage design also creates an extension point: the interpretation call could use a different model from the tool selection call, enabling model routing by task type with no architectural changes. Throughout the system, standard OpenAI-compatible API calls are used. Swapping the underlying model requires changing one configuration variable.

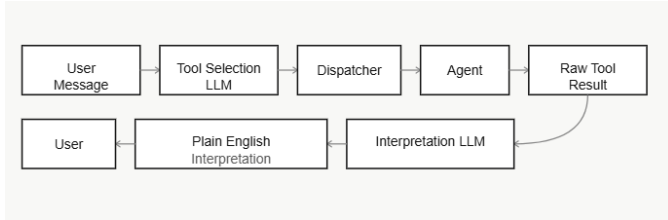


Fig. 3. Full workflow for a single user query. The sequence shows message intake, documentation retrieval, tool selection (first model call), agent execution, and plain English interpretation (second model call), with raw output displayed before interpretation

IV. EVALUATION

The system is a framework as much as an application. The relevant evaluation questions are therefore architectural: does the routing layer reliably direct queries to the correct agent? Does retrieval return domain-relevant context? Does the fallback mechanism hold up under failure conditions? Does the system preserve conversational context across turns?

These questions have observable answers, and the evaluation tested each one directly. Routing was verified across all seven agents using natural language queries covering every tool. Retrieval quality was assessed by inspecting returned chunks for domain relevance and comparing interpretation quality with and without retrieval active. Fallback behaviour was exercised throughout the entire evaluation run without a live

database connection. Conversational context was tested across multi-turn interactions involving cross-session references. The overall workflow and the models understanding is show-cased here.

A. Tool Routing

TABLE III
TOOL ROUTING RESULTS

Query	Tool selected by model
Show me the nodes in grid001	get_grid_data
Delete grid001	delete_grid
Show historical measurements for node PT in grid001	get_historical_data
What is the phase identification score for user001 on grid001?	get_phase_score
Submit topology guesses for grid001 with user id user001	submit_topology_guesses
What is the topology discovery score for user001 on grid001?	get_topology_score
Retrieve state estimation training data with no noise	get_training_state_data
What is the voltage control score for user001 on grid001?	get_voltage_control_score
Run power flow on grid001	run_power_flow
Show me voltage magnitudes at all nodes in grid001	get_power_flow_results
Get clean training data for phase identification	get_training_phase_data
Get hard difficulty phase identification test data	get_phase_identification_data
Get clean topology training data	get_training_topology_data
Get state estimation training data with no noise, medium observability	get_training_state_data
Get training voltage control data for overvoltages with voltage limit 230	get_training_voltage_control_data

Natural language queries covering all seven agents were submitted. The system routed correctly across all tested phrasings. The test queries were designed to reflect realistic engineer usage patterns. Routing reliability depended directly on the quality of tool schema descriptions. Action-oriented descriptions written in the style of what an engineer wants to accomplish produced more reliable routing than descriptions framed as API documentation. This was a consistent finding rather than an occasional edge case, and it holds as a design principle for any domain deployment of the framework.

B. Workflow Awareness

The system recognised workflow ordering across multi-turn interactions. The model correctly identified that certain operations depend on prior setup steps and did not attempt workflows such as power flow analysis before the required data was registered

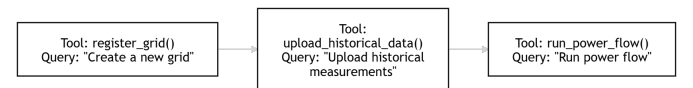


Fig. 4. Example of expected workflow behaviour. The model correctly identifies that a power flow cannot be run before the grid data is registered, and informs the engineer of the required prior steps

This behaviour came from tool schema descriptions that specify prerequisites and retrieved documentation that includes cross-agent workflow guidance. No explicit ordering logic is hard-coded anywhere.

Multi-turn conversational context was preserved correctly. Queries such as “run that again on a different grid” resolved correctly using session-level message history combined with per-agent state tracking.

C. Retrieval Quality

The per-agent collection design kept retrieval scoped to documentation relevant to the executing tool. Retrieved chunks included parameter explanations, domain interpretation guidance, and workflow context appropriate to each query. The improvement over operating without retrieval was most noticeable for queries requiring numerical domain knowledge: voltage thresholds in physical units, what phase identification accuracy scores of 0.33 versus 1.0 represent, topology scoring fields such as `incorrect_extra` and `missed`, and state estimation scoring based on sum of squared errors.

Example Query: “What is a good phase identification score?”

Without RAG: Model responded “I don’t have specific information about phase identification scoring.”

With per-collection RAG: Retrieved chunk from `phase_agent` documentation: “Scores range 0–1; 0.33 is random baseline, 0.7+ indicates usable performance, 1.0 is perfect.” Model then produced correct interpretation.

D. Fallback Behaviour

The evaluation ran entirely on mock data. No live database connection was configured at any point during testing. All seven agents activated their fallback behaviour and the full pipeline continued working. Mock responses exist for every agent and each is structurally identical to a real API response. The model produced coherent, domain-appropriate interpretations throughout. No code changes are needed to move to live data; the transition requires only setting the database connection environment variable.

E. Limitations

The system was evaluated without live data. Operational validation at scale has not been done, and is the appropriate next step before production deployment.

The documentation underpinning retrieval was generated as a structured starting point and has not been reviewed by domain electrical engineers. Numerical thresholds, domain heuristics, and edge case guidance are unverified. Since interpretation quality is directly bounded by documentation quality, expert review of the documentation is the single highest-leverage improvement available. The chunk size and overlap parameters 600 words with 100-word overlap were set as a reasonable starting point and have not been benchmarked; tuning these may improve retrieval performance.

Early in development, the model occasionally produced malformed argument structures for tools with complex parameter types. Introducing retrieval reduced the frequency of those

failures, which means retrieval quality is doing work beyond information provision. Degradation in retrieval risks bringing those failures back.

The framework is already seeing active extension by other developers. New agents are currently being developed within the same architecture to serve as literature assistants for each benchmark use case, demonstrating the transferability of the agent pattern to new domains without modifications to the core infrastructure

None of these limitations require architectural changes to address. Live data requires environment configuration. Validated documentation requires authorship effort. Chunk tuning requires a benchmarking pass. All of it is extension work.

V. CONCLUSION

This system is a reusable architectural pattern for giving domain experts natural language access to APIs they would otherwise need developer mediation to use. Gridarena is the domain it was built and validated for. The architecture is not specific to it.

The five layer design, the uniform agent pattern, per-collection RAG, two-stage response format, and structured mock fallback are all domain independent. Adding a new agent is a seven-step process. Swapping the underlying model is a one-line configuration change. Improving documentation quality improves interpretation quality everywhere that documentation is retrieved with no changes to any other layer. Deploying the framework against a new API requires writing domain documentation and agent wrappers; the routing, retrieval, conversation, and fallback infrastructure is already there.

The gaps are real. The system has not been validated on live data. The documentation has not been reviewed by domain electrical engineers, which means the interpretation quality for gridarena is limited by unverified content. The author conducted the evaluation. None of these require rethinking the design. They are content and validation work, not architectural problems. The path from here to a production deployment on any target API is one of filling in that content, not revisiting the structure.

REFERENCES

- [1] Agenta, “The guide to structured outputs and function calling with LLMs,” [Online]. Available: <https://agenta.ai/blog/the-guide-to-structured-outputs-and-function-calling-with-llms>
- [2] Digital Applied, “AI function calling guide: OpenAI, Anthropic, Google,” [Online]. Available: <https://www.digitalapplied.com/blog/ai-function-calling-guide-openai-anthropic-google>
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020. [Online]. Available: <https://www.researchgate.net/publication/341639856>
- [4] OpenAI, “Function calling in the OpenAI API,” [Online]. Available: <https://help.openai.com/en/articles/8555517-function-calling-in-the-openai-api>
- [5] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large language model connected with massive APIs,” arXiv:2305.15334, 2023. [Online]. Available: <https://arxiv.org/abs/2305.15334>
- [6] Y. Song, W. Xiong, D. Zhu, W. Wu, H. Qian, M. Song, and S. Li, “RestGPT: Connecting large language models with real-world RESTful APIs,” 2023. [Online]. Available: <https://restgpt.github.io>
- [7] Chroma Core, “Chroma: Search infrastructure for AI,” GitHub repository, 2024. [Online]. Available: <https://github.com/chroma-core/chroma>